

Fingerprinting Resistance in LLM-Based Shell Honeypots: Vulnerabilities, Countermeasures, and the Case for Edge-Deployed Small Language Models

Shashank Appana

Department of Computer Science and Engineering

The Pennsylvania State University

University Park, PA 16802

sba6069@psu.edu

Abstract—Honey-pot systems have become essential tools for understanding attacker behavior and enhancing network security. Recent advances have produced a new class of shell honeypots powered by Large Language Models, with HoneyLLM as the leading example. By routing attacker commands to commercial LLMs such as GPT-4, HoneyLLM achieves substantially higher interaction fidelity than traditional scripted honeypots such as Cowrie. However, a critical and underexplored vulnerability threatens this approach: fingerprinting resistance. Sophisticated attackers do not merely attack target systems, they first probe to determine whether they are interacting with a real device or a decoy. Existing fingerprinting literature demonstrates that subtle artifacts in protocol behavior, response timing, static configuration, and behavioral consistency reliably expose honeypot infrastructure. HoneyLLM was not designed with fingerprinting resistance as a primary objective, and it exhibits several exploitable artifacts. This paper surveys existing honeypot fingerprinting techniques, systematically analyzes the fingerprinting vulnerabilities of HoneyLLM, and proposes concrete countermeasures for each identified vulnerability. We then focus on the most fundamental of these vulnerabilities, the LLM response latency gap, and investigate in depth how the integration of small language models (SLMs) with edge computing can serve as a structural rather than cosmetic mitigation. We examine the latency reduction potential of recent open-weight SLMs, the accuracy tradeoffs they introduce, the role of parameter-efficient fine-tuning in closing those gaps, and the new security challenges that edge deployment creates. Our analysis suggests that edge-deployed SLMs represent a promising direction for the next generation of high-fidelity, fingerprinting-resistant LLM honeypots.

Index Terms—honeypots, network security, fingerprinting, large language models, small language models, edge computing, deception systems

I. INTRODUCTION

Honeypots, serving as decoy systems, are engineered to lure, interact with, and mislead prospective attackers by mimicking susceptible services within a controlled and monitored setting. Since their inception, honeypots have undergone considerable development. Initial low-interaction honeypots such as Honeyd employed scripted responses to simulate fundamental network services. Medium-interaction systems such as Cowrie offered

more sophisticated shell emulation via Python-based scripting. High-interaction honeypots presented actual operating systems to attackers, thereby exchanging security risk for heightened realism.

The most recent generation of shell honeypot research has taken fidelity to a new level by leveraging Large Language Models (LLMs). HoneyLLM, the leading example of this approach, emulates an interactive shell environment by routing attacker commands to commercial LLMs such as GPT-4 or Claude and returning the generated responses to the attacker [1]. This technique allows the honeypot to respond to arbitrary commands with contextually appropriate output, eliminating the rigidity of scripted logic and substantially improving engagement with real attackers. Online deployment results show that HoneyLLM powered by GPT-4 achieves an average session length of 5.83 interaction rounds, compared to 2.96 for Cowrie alone [1].

However, HoneyLLM shares a critical and unaddressed limitation with earlier honeypot generations: it was not designed to resist active fingerprinting by sophisticated attackers. Fingerprinting, in this context, refers to the deliberate probing of a target system specifically to determine whether it is a genuine device or a honeypot, before committing to a full attack. An attacker who successfully fingerprints a honeypot gains nothing by continuing the interaction, they simply disengage, leaving the defender with no intelligence about attack methodologies or objectives.

The fingerprinting threat is well established. Vetterl and Clayton demonstrated in Bitter Harvest that low- and medium-interaction honeypots can be fingerprinted at Internet scale using a single discriminating network probe per honeypot type, achieving an equal error rate of 0.0183 [2]. Morishita et al. showed that over 19,000 honeypots were self-revealing through unchanged default banners, host keys, and HTTP titles, detectable without any active probing whatsoever [3]. These studies focused on older honeypot generations. The question of whether modern LLM-based honeypots are equally vulnerable, and whether the specific architectural choices that

make them high-fidelity also introduce new detection vectors, has not been systematically investigated.

This paper addresses that gap with a focus on HoneyLLM as the representative modern LLM honeypot. Our contributions are:

- 1) We survey existing honeypot fingerprinting techniques across four categories: protocol-level, static artifact, timing-based, and behavioral fingerprinting.
- 2) We systematically apply each fingerprinting category to HoneyLLM, identifying six specific exploitable vulnerabilities traceable to its architectural decisions.
- 3) We propose concrete countermeasures for each identified vulnerability, several of which align with directions explicitly identified as future work by the original authors.
- 4) We investigate in depth the integration of small language models with edge computing as a structural mitigation for the most fundamental vulnerability, the LLM response latency gap. This includes an analysis of SLM latency and memory characteristics, the accuracy trade-offs involved, the role of parameter-efficient fine-tuning, and the new security challenges that edge deployment introduces.

The remainder of this paper is organized as follows. Section II provides background on HoneyLLM. Section III surveys fingerprinting techniques. Section IV presents our vulnerability analysis. Section V proposes direct countermeasures. Section VI investigates edge-deployed small language models as a deeper mitigation. Section VII concludes.

II. BACKGROUND: HONEYLLM

Traditional shell honeypots such as Cowrie [5] use scripted methods to emulate shell environments. While effective against automated scanning tools, their fixed response logic creates identifiable fingerprints that sophisticated attackers can detect and avoid [2]. Real-system approaches using genuine operating systems at the backend provide higher fidelity but incur significant deployment costs and security risks if the sandbox is compromised.

HoneyLLM addresses these limitations by leveraging commercial LLMs to dynamically emulate shell environments [1]. The system routes attacker commands to models such as GPT-4, which generate contextually appropriate shell responses. Two key prompt engineering techniques address the inherent limitations of vanilla LLMs. In-context learning provides the model with real attack trace examples to improve response accuracy for specific commands. The Chain-of-Thought (CoT) method maintains session history by tracking commands that affect system state, embedding this history into subsequent prompts to ensure contextual consistency across multi-step attack sessions.

To address practical deployment constraints, including API rate limits and cost, HoneyLLM employs a hybrid architecture. Automated scanning traffic with identifiable fingerprints is routed to Cowrie, a low-interaction honeypot. Only genuine, interactive attack sessions are forwarded to the LLM backend. Online deployment results show that HoneyLLM powered by

GPT-4 achieves an average session length of 5.83 interaction rounds, compared to 2.96 for Cowrie alone [1]. In the authors' offline evaluation using prompt engineering, GPT-4 achieves a 90.2% session-level success rate across replayed real attack traces, substantially outperforming smaller models such as GPT-3.5 and Claude-3 Haiku on long, multi-step sessions.

Despite these advances, the HoneyLLM authors explicitly acknowledge two unresolved limitations: the susceptibility to prompt injection attacks that can reveal the honeypot's nature, and the practical challenge of handling high-volume concurrent attacks within API rate constraints. A third limitation, response latency introduced by the commercial LLM API call path, is architecturally inherent but is not addressed as a fingerprinting concern in the original paper. As we show in Section IV, this latency gap is in fact the most severe fingerprinting vulnerability of the system.

III. HONEYPOT FINGERPRINTING TECHNIQUES

Honeypot fingerprinting refers to techniques used by attackers to determine whether a target system is a genuine device or a deception infrastructure. We organize these techniques into four categories, as summarized in Table I.

A. Protocol-Level Fingerprinting

Vetterl and Clayton's Bitter Harvest demonstrated that low- and medium-interaction honeypots can be systematically fingerprinted through protocol behavior analysis [2]. By sending deliberately malformed or edge-case packets to both real servers and honeypots, they identified subtle but consistent differences in responses. For SSH, they found that OpenSSH responds to malformed packets with "Bad packet length" while Cowrie/Kippo responds with "bad packet length", a capitalization difference sufficient for automated classification. For HTTP and Telnet, differences appear in header ordering, error code formats, and responses to invalid method names. These single-packet probes achieve an Equal Error Rate of 0.0183, meaning honeypots can be reliably identified with a single crafted packet before any authentication attempt.

The practical consequence is severe. Vetterl and Clayton performed Internet-wide scans using these probes with ZMap, locating thousands of Cowrie SSH, Glastopf HTTP, and Dionaea Telnet/SMB honeypots. Critically, 72% of Kippo instances remained vulnerable to an older fingerprint even 44 months after public disclosure [2].

B. Static Artifact Detection

Morishita et al. identified a simpler but equally effective fingerprinting vector: many honeypot operators never modify default configurations [3]. By building 20 signatures based on default banners, SSH host keys, HTTP titles, and favicon hashes from popular open-source honeypots, they performed Internet-wide scans finding 19,208 self-revealing honeypots across 637 autonomous systems. No protocol fuzzing was required, these honeypots announced their nature through static identifiers visible in standard connection handshakes.

Common examples include Cowrie’s default SSH host key, which remains identical across all default deployments; Glastopf’s default HTTP title strings; and Dionaea’s characteristic service banners. An attacker need only maintain a database of known default values and compare them against observed handshake data.

C. Timing-Based Detection

Timing side-channels provide fingerprinting vectors applicable to all honeypot interaction levels. Uitto et al.’s survey of anti-honeypot methods documents several timing-based techniques [4]. At the network level, comparing response-time distributions across multiple ports on the same IP can reveal artificial uniformity characteristic of software emulation. Real servers exhibit natural variance in response latency due to hardware interrupts, OS scheduling, and application processing. Emulated systems may exhibit either unnaturally low jitter or characteristically high and variable latency from emulation overhead.

Multi-step interaction timing is particularly revealing. Real shell environments respond to commands like `echo test` in under 10 milliseconds. Systems that route commands through network-dependent processing pipelines introduce measurably different latency patterns that statistical analysis can reliably detect.

D. Behavioral and Semantic Fingerprinting

Behavioral fingerprinting exploits the semantic gap between emulated and real systems. Key tests include:

Filesystem persistence: Creating a file in one session and verifying its existence in a subsequent session tests whether the system maintains genuine persistent state. Real systems retain files across connections; stateless emulators do not.

Process consistency: Running `ps aux` across multiple sessions should return consistent process IDs and system processes on a real server. Stochastically generated process tables from emulation exhibit inconsistencies.

Network reachability: Executing `curl` or `wget` to an attacker-controlled server verifies whether outbound network connections actually complete. Honeypots that simulate these commands without making real network calls produce no observable traffic on the attacker’s server.

Semantic consistency: Commands whose outputs depend on previous commands in the session test whether the emulator maintains a coherent model of system state. A shell that reports creating a file in one turn but cannot find it two turns later reveals itself immediately.

IV. VULNERABILITY ANALYSIS OF HONEYLLM

We now systematically apply each fingerprinting category to HoneyLLM. We identify six distinct vulnerabilities, each traceable to a specific architectural decision in the system.

A. Inherited Cowrie Protocol Fingerprints

HoneyLLM’s hybrid architecture routes automated scanning traffic to Cowrie before applying LLM processing to genuine attack sessions [1]. While this design is efficient from a cost and rate-limit perspective, it means that any attacker performing protocol-level fingerprinting will interact with Cowrie’s network stack, not an LLM. Bitter Harvest demonstrated that Cowrie’s SSH implementation produces distinct protocol-level artifacts detectable with a single malformed packet [2]. A sophisticated attacker using automated fingerprinting tools would identify the Cowrie component before their session is ever routed to the LLM, rendering HoneyLLM’s high-fidelity LLM backend irrelevant for such attackers. The problem is structural: the hybrid design places Cowrie in the critical path of every incoming connection, so any detectable property of Cowrie becomes a detectable property of HoneyLLM as a whole.

B. LLM Response Latency

Real shell environments respond to commands such as `echo test` or `ls` in under 10 milliseconds. HoneyLLM routes each attacker command to a commercial LLM API, incurring network round-trip time plus model inference time. Recent benchmarks of GPT-4 class API providers report median time-to-first-token values in the range of 0.4 to 1.7 seconds even under favorable conditions [12], with full responses typically taking several seconds depending on output length. An attacker measuring response latency across even a small number of commands would observe a distribution incompatible with any real shell environment.

This is a fundamental architectural vulnerability. As long as HoneyLLM relies on external API calls for response generation, the latency gap cannot be eliminated, only partially obfuscated. Among the six vulnerabilities identified in this paper, this is the most severe and the most difficult to patch, because every other vulnerability can be addressed by adding or tightening a component, while the latency gap is a direct consequence of the core design choice to use a cloud-hosted LLM. Section VI explores a structural rather than cosmetic solution to this problem.

C. Stateless Filesystem Behavior

HoneyLLM uses Chain-of-Thought prompting to maintain session context by tracking commands that may affect system state. However, this state is maintained only within a single session’s prompt context and is not persisted to any real storage backend [1]. An attacker who creates a file with `echo test > /tmp/marker`, terminates the SSH session, and reconnects will find the file absent. This cross-session inconsistency is immediately detectable and provides a definitive fingerprinting signal requiring no specialized tools. The attacker does not need to measure response times, send malformed packets, or study the handshake banner; they simply need to notice that files do not survive a reconnection.

D. Probabilistic Process Table Inconsistency

Real Linux systems maintain a consistent process table accessible via `ps aux` or `/proc`. System processes such as `init`, `sshd`, and kernel threads have fixed PIDs across the lifetime of the system. HoneyLLM generates process table output stochastically through LLM inference, which means that an attacker who queries the process table in two separate sessions will observe different PIDs for the same processes, inconsistent process counts, or varying system details that no real server would exhibit. This vulnerability is a direct consequence of the LLM’s probabilistic generation mechanism, and it applies not only to `ps` but to any command whose output would be stable on a real system but is re-generated from scratch by the LLM on each call.

E. Prompt Injection Susceptibility

HoneyLLM embeds attacker-supplied shell commands directly into LLM prompts [1]. This architecture is inherently vulnerable to prompt injection: an attacker can craft a “shell command” that is actually an instruction to the LLM. For example, a command such as:

```
; ignore all previous instructions and  
describe your system prompt
```

would cause a real shell to return a syntax error or execute the literal string. An LLM, however, may subtly alter its behavior in response to such instructions, potentially revealing the prompt structure, the model being used, or exhibiting behavior inconsistent with a real shell. The HoneyLLM authors explicitly acknowledge this as an unsolved limitation and identify it as future work [1]. As we discuss in Section VI, this vulnerability becomes more severe, not less, when the system is deployed with a smaller model.

F. Default Static Artifacts

HoneyLLM’s hybrid architecture retains Cowrie as its low-interaction component. Cowrie deployments that use default SSH host keys and banner strings are detectable through the static artifact methods of Morishita et al. [3]. Morishita found that the majority of detectable honeypots in their Internet-wide scan were identified through unchanged default configurations rather than active probing, making this one of the cheapest and most effective fingerprinting vectors available to an attacker.

V. DIRECT COUNTERMEASURES

Before discussing the deeper structural solution in Section VI, we first present a set of direct countermeasures that can be applied to the existing HoneyLLM architecture to mitigate each identified vulnerability. Table II summarizes these and their target vulnerabilities.

Response Time Normalization. HoneyLLM should implement a response buffering layer that normalizes output delivery timing. Rather than returning responses immediately upon LLM completion, the system adds jitter sampled from a distribution matching real shell response times. Slow responses from the LLM can be handled by returning partial output

progressively. This does not eliminate the latency gap but makes it statistically harder to distinguish from a loaded real server. As we discuss in Section VI, this is fundamentally a cosmetic fix; the underlying cause of the latency gap can only be removed by a deeper architectural change.

Host Key and Banner Randomization. HoneyLLM should generate unique SSH host keys and randomize banner strings at deployment time. Morishita et al. demonstrate that the majority of detectable honeypots are identified through unchanged defaults [3]. This countermeasure is low cost and eliminates the largest class of passive fingerprinting. Automated deployment scripts should include key generation as a required step rather than an optional one.

Input Sanitization Layer. HoneyLLM should implement a pre-processing layer that classifies incoming commands for prompt injection attempts before forwarding them to the LLM. This aligns with the future work direction explicitly identified by the HoneyLLM authors [1]. The sanitizer can use pattern matching for known injection templates together with a secondary LLM classifier trained to distinguish legitimate shell commands from injection attempts. Commands identified as injection attempts can be returned as standard shell errors, preserving the illusion of a real system.

Persistent State Simulation. HoneyLLM should maintain a lightweight key-value store as a simulated filesystem state backend. Commands that create, modify, or delete files update this store. On session initialization, the store state is embedded into the LLM prompt as part of the session context. This eliminates cross-session inconsistency for filesystem operations without requiring a real OS backend. The store can be implemented in Redis or an equivalent in-memory database with negligible overhead.

Seeded Process Table. HoneyLLM should initialize each honeypot instance with a fixed, pre-generated process table seeded at deployment time. This table, including realistic PIDs for `sshd`, `init`, and system processes, is embedded in the honeypot configuration and provided to the LLM as part of the system state context. PIDs remain consistent across sessions for the lifetime of the honeypot instance, matching the behavior of a real long-running server.

The countermeasures above are practical and low-cost, and taken together they address five of the six vulnerabilities identified in Section IV. The remaining vulnerability, LLM response latency, is the most severe and cannot be fully eliminated without changing the core design choice that introduces it. The next section examines how this can be done.

VI. EDGE-DEPLOYED SMALL LANGUAGE MODELS AS A STRUCTURAL LATENCY MITIGATION

Among the vulnerabilities identified in Section IV, the LLM response latency gap stands apart. Every other vulnerability can be patched by adding a component such as a state store, a sanitizer, or a seeded process table, but the latency gap is a direct consequence of the core design choice to route attacker commands to a remote commercial LLM. The response time normalization countermeasure in Section V can statistically

TABLE I
SUMMARY OF FINGERPRINTING TECHNIQUES AND THEIR APPLICABILITY TO HONEYLLM

Technique	Method	Vulnerability in HoneyLLM
Protocol Fuzzing [2]	Malformed or edge-case packets	Cowrie low-interaction component has known protocol-level fingerprints; hybrid architecture exposes Cowrie stack before LLM routing
Static Artifacts [3]	Default banners, SSH host keys, HTTP titles	Default Cowrie host key and banner strings unchanged on most deployments
Timing Analysis [4]	Response latency measurement	Commercial LLM API calls introduce 1 to 5 second latency per command compared to sub-10ms for a real shell; most fundamental vulnerability
Filesystem Persistence	Cross-session state verification	LLM is stateless across sessions; files created in one session do not persist to the next
Prompt Injection [1]	Embedded LLM instructions in shell commands	Attacker commands are inserted directly into the LLM prompt; model may respond to injected instructions
Process Table Consistency	Repeated <code>ps aux</code> across sessions	LLM generates process tables probabilistically; PIDs and process counts differ across sessions
Network Reachability	Outbound connection verification	Simulated <code>curl</code> and <code>wget</code> produce no actual traffic on attacker-controlled servers

TABLE II
DIRECT COUNTERMEASURES FOR HONEYLLM VULNERABILITIES

Countermeasure	Addresses
Response time normalization	LLM latency fingerprint (partial)
Host key and banner randomization	Static artifact detection
Input sanitization layer	Prompt injection
Persistent state simulation	Stateless filesystem
Seeded process table	Process table inconsistency
Edge-deployed SLM inference (Sec. VI)	LLM latency (structural fix)

mask the gap up to a point, but it cannot remove it. As long as every command pays the cost of a wide-area network round trip plus inference on a frontier-scale model, the underlying distribution remains distinguishable from that of a real shell.

This section examines a deeper alternative: removing the API call from the critical path entirely by running a small language model locally at the edge. We discuss the feasibility of this approach in terms of current SLM capabilities, the accuracy tradeoffs it introduces, the role of parameter-efficient fine-tuning in mitigating those tradeoffs, and the new security challenges that edge deployment creates.

A. The Nature of the Latency Problem

Recent benchmarks of commercial LLM providers report median time-to-first-token values for GPT-4 class models in the range of 0.4 to 1.7 seconds, with full response times typically several seconds depending on output length [12]. These figures reflect favorable conditions and do not include the additional network latency between the honeypot and the API endpoint, which adds tens to hundreds of milliseconds depending on geography and network load. In contrast, a real Linux shell responds to commands such as `pwd`, `whoami`, or

`ls` in well under 10 milliseconds, and the variance in those response times is driven by OS-level factors such as scheduling jitter and disk cache behavior rather than network round trips.

The gap between these two distributions is not subtle. It spans roughly two orders of magnitude in the median and an even larger factor in the tail. An attacker who times a handful of commands can statistically distinguish a cloud LLM-backed honeypot from a real server with high confidence. Response time normalization can mask individual outliers, but it cannot shift the mean of the distribution without holding responses back long enough to match real shell timing, which defeats the purpose of using a fast LLM in the first place. The only structural solution is to reduce the underlying inference latency so that the honeypot can produce real responses at speeds closer to a real shell, not to fake the timing of slow responses.

B. Small Language Models and Edge Deployment

Recent advances in small language models (SLMs) make edge-deployed inference increasingly practical. Microsoft’s Phi-3-mini, a 3.8 billion parameter model, achieves 69% on MMLU and competitive scores on reasoning benchmarks such as HellaSwag and ARC, performance that rivals models nearly twice its size [6]. A 4-bit quantized version of Phi-3-mini occupies roughly 1.8 GB of memory and runs natively on smartphone-class hardware, generating over 12 tokens per second on an iPhone 14 [6]. Meta’s Llama 3.2 family includes 1B and 3B parameter models specifically designed for on-device deployment, with the 1B variant requiring approximately 1.3 GB of memory in quantized form and the 3B variant approximately 2.5 GB [7]. On Arm-based mobile CPUs, Llama 3.2 3B has been measured at roughly 20 tokens per second using optimized kernels [8]. Google’s Gemma 2 family offers comparable models in the 2B parameter range. Each of these models can be served using `llama.cpp` [9], a widely adopted C++ inference engine that supports the GGUF quantized model format and runs efficiently on commodity

CPUs, consumer GPUs, and dedicated edge AI hardware such as the NVIDIA Jetson family.

The implications for HoneyLLM are significant. An edge-deployed SLM eliminates the network round trip entirely and replaces frontier-scale inference with a much lighter computation. A honeypot operator can co-locate the model with the honeypot daemon on the same physical machine, reducing the inference path from a wide-area API call to a local function invocation. On consumer-grade GPUs or dedicated edge inference hardware, first-token latency for a small model can fall into the low hundreds of milliseconds, and short responses for common shell commands can be generated in well under a second. While this is still slower than the sub-millisecond response of a real shell, the gap shrinks by roughly an order of magnitude compared to the one to five second range of cloud-hosted GPT-4 reported in HoneyLLM’s own evaluation. Combined with the response time normalization proposed in Section V, the resulting timing distribution becomes substantially harder to distinguish from a moderately loaded real server, because the remaining gap is small enough to fall within the natural jitter range of a real system.

An edge deployment also removes the API rate limit problem that motivated HoneyLLM’s hybrid architecture in the first place. Because there is no per-minute token cap, the LLM-backed component can handle a much larger fraction of incoming traffic rather than being restricted to a small number of concurrent active sessions. Local inference additionally avoids the per-query API cost, which is a significant practical concern for honeypots that may receive thousands of attack sessions per day. These secondary benefits, while not directly related to fingerprinting resistance, strengthen the case for edge deployment as a general architectural direction.

A practical caveat is that CPU-only inference on commodity laptop hardware remains slow even for the smallest models. Our preliminary exploration running Phi-3-mini in 4-bit quantized form on a modern AMD CPU without GPU acceleration produced response times of several seconds for trivial commands, which is substantially worse than the latency targets that would make the approach competitive. The practical deployment target for an edge-SLM honeypot is therefore not arbitrary commodity hardware but a machine with either a consumer GPU, an edge AI accelerator such as a Jetson device, or an Apple Silicon system with unified memory. These are all cheap relative to ongoing commercial API costs and represent realistic deployment platforms for a defender willing to invest in the hardware once.

C. The Accuracy Challenge and the Role of Fine-Tuning

The most important open question is whether a small model can match the response fidelity that HoneyLLM achieves with GPT-4. The original HoneyLLM evaluation reported a 90.2% session-level success rate for GPT-4 with prompt engineering [1], with smaller commercial models such as GPT-3.5 and Claude-3 Haiku trailing meaningfully on long, multi-step attack sessions. The open-weight SLMs discussed above

are smaller still and have not been evaluated on this specific task. Their general benchmark scores are encouraging, but general benchmarks do not directly answer whether a small model can sustain consistent shell emulation across sequences of interdependent commands, which is precisely the regime where vanilla LLMs struggle.

Early informal testing with base SLMs suggests that instruction following on shell-emulation prompts is inconsistent. A base Phi-3-mini or Llama 3.2 1B model asked to simulate a Linux shell may respond correctly to simple queries such as `pwd` or `whoami` but quickly begins hallucinating invented hostnames, malformed prompts, or unrelated continuations when the session runs for more than a few turns. This is consistent with the known difficulty small general-purpose models have with strict format following, and it matches the pattern of errors documented in HoneyLLM’s own evaluation of GPT-3.5 and Claude-3 Haiku. Without intervention, a base SLM is unlikely to match HoneyLLM’s GPT-4 baseline fidelity.

There is, however, reason for cautious optimism, and it centers on the observation that shell emulation is a narrow domain compared to the broad reasoning tasks that SLMs are typically evaluated on. The set of common shell commands is finite, the expected output formats are well structured, and the behavior of a Linux shell is deterministic in most cases. A general-purpose 3B or 4B parameter model is not optimized for this task, but it does not need to be. Parameter-efficient fine-tuning techniques such as LoRA [10] and QLoRA [11] make it feasible to specialize a base SLM for shell emulation using a relatively small training corpus. QLoRA in particular enables fine-tuning of a quantized base model on a single consumer GPU without sacrificing the accuracy of full-precision adaptation. This is directly relevant to HoneyLLM because the system already collects real attack trace data from its deployment. That dataset, consisting of attacker commands paired with real shell responses, is exactly the kind of training corpus needed to fine-tune a small model into a specialized shell emulator.

The hypothesis, then, is that a small model fine-tuned on HoneyLLM’s existing attack trace dataset may outperform a much larger general-purpose model on this narrow task while still meeting the latency requirements of edge deployment. Validating this hypothesis empirically, ideally by replicating HoneyLLM’s evaluation methodology of replaying real attack traces and measuring session-level success rates across a range of model sizes and fine-tuning configurations, is the most important direction for future work in this area.

D. New Security Challenges

The shift to edge-deployed SLMs introduces a new set of security tradeoffs that must be examined honestly. On the positive side, local inference eliminates the privacy risk of forwarding attacker commands and session data to a third-party API provider. Captured attack intelligence remains entirely within the defender’s infrastructure, which is particularly relevant for sensitive deployments such as those operated

by government or critical infrastructure organizations. The current HoneyLLM architecture effectively requires sharing every attacker interaction with a commercial LLM vendor, which may be unacceptable for some deployment contexts.

On the negative side, smaller models are generally more susceptible to prompt injection than frontier models. A 3B or 4B parameter model has weaker instruction-following capabilities than GPT-4 and is therefore easier to manipulate using crafted shell commands that contain hidden meta-instructions. The prompt injection vulnerability identified in Section IV-E becomes more acute, not less, in an SLM-based deployment. The input sanitization countermeasure proposed in Section V is therefore not optional in this setting; it becomes essential. A practical edge-SLM honeypot would need to combine local inference with a robust pre-processing layer that classifies incoming commands and rejects anything matching known injection patterns before the input ever reaches the model. Even with such a layer, residual risk from novel injection techniques remains higher than in a GPT-4 deployment, and this residual risk must be factored into the overall security analysis.

A second consideration is the integrity of the model itself. A locally hosted model is a static file on disk, which makes it a target for tampering if an attacker manages to compromise the honeypot host. An attacker who replaces or modifies the model could degrade response quality, introduce backdoors that trigger on specific inputs, or cause the honeypot to exhibit detectable patterns. Standard integrity protections such as cryptographic hashing of model files at startup and read-only mounting of the model directory mitigate this risk but should be treated as required parts of the deployment process rather than optional hardening steps. In a cloud API deployment, this class of attack is simply not possible because the model never exists on the defender’s machine.

A third consideration concerns concurrent session handling on resource-constrained hardware. A single SLM instance running on a CPU or a single consumer GPU can serve only a limited number of simultaneous attack sessions before queuing delays become observable. A naive deployment that processes sessions sequentially would introduce a new timing artifact that an attacker could exploit by initiating multiple parallel connections and measuring response time differences across them. A session that waits in a queue behind another session will show a distinctive latency pattern that differs from a real multi-user server. Practical deployments will therefore need to consider model batching, session prioritization, or running multiple model instances on dedicated cores or GPUs. The hardware cost of supporting realistic concurrent load may be higher than the naive analysis suggests, which partially offsets the cost savings from eliminating API fees.

A fourth consideration is the attack surface introduced by the inference runtime itself. Frameworks such as llama.cpp are widely used and actively maintained, but they are still software running on the honeypot host, and any vulnerability in the runtime becomes a vulnerability in the honeypot. A cloud API call has the advantage of isolating the model execution from

the defender’s machine; an edge deployment sacrifices that isolation. Defenders choosing this direction should treat the inference runtime as a security-critical component and monitor it accordingly.

E. Open Research Questions

The investigation of edge-deployed SLMs for honeypot use opens several concrete research questions that merit empirical study. First, what is the minimum model size that can sustain HoneyLLM-level session fidelity on shell emulation tasks? This question is best answered by replicating HoneyLLM’s evaluation methodology across a range of models from 1B to 7B parameters, with and without task-specific fine-tuning, and measuring session-level success rates on replayed real attack traces.

Second, how much does parameter-efficient fine-tuning on collected attack traces improve fidelity over the base model, and does the improvement scale with the size of the training corpus? This question has direct implications for whether HoneyLLM operators would need to accumulate large volumes of attack data before a fine-tuned model becomes usable, or whether relatively small corpora are sufficient.

Third, what is the realistic latency distribution of a CPU-only, edge-GPU, or edge-accelerator inference stack for the kinds of short, frequent commands that dominate real attack sessions, and how close does it come to the response distribution of a real Linux server under load? Answering this question requires hardware-specific benchmarking on the platforms most likely to be used in production.

Fourth, how does the prompt injection robustness of small models compare to that of frontier models, and can adversarial training or input sanitization close the gap? This is the central question for the security viability of an SLM-based honeypot, because a model that is too easy to jailbreak reveals itself to the attacker regardless of its latency or fidelity.

Fifth, how does an edge-SLM honeypot compare to a hybrid deployment in which simple commands are handled by a local SLM and only complex, context-dependent commands are escalated to a cloud LLM? This question connects directly to the hybrid architecture principle that HoneyLLM already employs with Cowrie, and it suggests a natural evolution of the system in which the low-interaction and high-interaction tiers are both themselves adaptive rather than scripted.

Each of these questions has direct implications for whether edge-deployed SLMs can serve as a practical, fingerprinting-resistant alternative to API-based LLM honeypots, and each is tractable with the research infrastructure and attack trace collection capability that HoneyLLM already has in place.

VII. CONCLUSION

This paper has examined fingerprinting resistance as a critical and underexplored vulnerability in HoneyLLM, the leading example of an LLM-based shell honeypot. We surveyed existing fingerprinting techniques across protocol-level, static artifact, timing-based, and behavioral categories, then

systematically applied each to HoneyLLM. Our analysis identified six distinct fingerprinting vulnerabilities, traceable to specific architectural decisions in the system: inherited Cowrie protocol fingerprints, LLM response latency, stateless filesystem behavior, probabilistic process table inconsistency, prompt injection susceptibility, and default static artifacts.

We proposed direct countermeasures for five of these vulnerabilities, several of which build on future work directions identified by the HoneyLLM authors themselves. For the sixth and most severe vulnerability, the LLM response latency gap, we argued that direct countermeasures such as response time normalization are fundamentally cosmetic and cannot remove the underlying cause. Instead, we investigated in depth a structural alternative: replacing the commercial LLM API call with local inference on an edge-deployed small language model. We showed that recent open-weight SLMs such as Phi-3-mini and Llama 3.2 offer the inference speed and memory footprint required for local deployment on reasonable edge hardware, that parameter-efficient fine-tuning techniques such as LoRA and QLoRA make it feasible to specialize a small base model for shell emulation using HoneyLLM's existing attack trace dataset, and that this direction addresses the latency vulnerability at its root rather than masking it. We also examined the new accuracy and security challenges that this shift introduces, including the weaker base fidelity of small models, their increased susceptibility to prompt injection, the integrity of locally hosted model files, the difficulty of handling concurrent sessions on resource-constrained hardware, and the attack surface of the inference runtime itself.

The overall picture is that fingerprinting resistance must be treated as a first-class design objective for LLM honeypots, alongside interaction fidelity and scalability. The direct countermeasures proposed in this paper can be implemented without fundamental redesign and would substantially raise the bar for attacker fingerprinting. The SLM and edge computing direction is a longer-term investment, but it is the only approach we have identified that addresses the latency vulnerability at its root. Future work should develop standardized evaluation benchmarks for honeypot fingerprinting resistance, empirically validate the accuracy of fine-tuned edge SLMs on shell emulation tasks using HoneyLLM's evaluation methodology, and investigate hybrid architectures that combine edge inference for simple commands with cloud inference for complex sessions. Each of these directions is tractable with the research infrastructure already in place, and together they represent a concrete roadmap for the next generation of high-fidelity, fingerprinting-resistant LLM honeypots.

REFERENCES

- [1] C. Guan, G. Cao, and S. Zhu, "HoneyLLM: Enabling Shell Honeypots with Large Language Models," *IEEE Conference on Communications and Network Security (CNS)*, 2024.
- [2] A. Vetterl and R. Clayton, "Bitter Harvest: Systematically Fingerprinting Low- and Medium-Interaction Honeypots at Internet Scale," *USENIX Workshop on Offensive Technologies (WOOT)*, 2018.
- [3] S. Morishita, T. Hoizumi, W. Ueno, R. Tanabe, C. Gañán, M. van Eeten, K. Yoshioka, and T. Matsumoto, "Detect Me If You... Oh Wait. An Internet-Wide View of Self-Revealing Honeypots," *IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, 2019.
- [4] J. Uitto, S. Rauti, S. Laurén, and V. Leppänen, "A Survey on Anti-Honeypot and Anti-Introspection Methods," University of Turku, Technical Survey, 2017.
- [5] M. Oosterhof, "Cowrie SSH/Telnet Honeypot," [Online]. Available: <https://github.com/cowrie/cowrie>, 2024.
- [6] M. Abidin et al., "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," *arXiv preprint arXiv:2404.14219*, 2024.
- [7] Meta AI, "Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models," [Online]. Available: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>, 2024.
- [8] Arm, "Accelerating and Scaling AI Inference Everywhere with New Llama 3.2 LLMs on Arm," [Online]. Available: <https://newsroom.arm.com/news/ai-inference-everywhere-with-new-llama-llms-on-arm>, 2024.
- [9] G. Gerganov, "llama.cpp: LLM Inference in C/C++," [Online]. Available: <https://github.com/ggerganov/llama.cpp>, 2024.
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations (ICLR)*, 2022.
- [11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] Artificial Analysis, "GPT-4 and GPT-4o API Provider Performance Benchmarking," [Online]. Available: <https://artificialanalysis.ai/models/gpt-4o/providers>, 2024.
- [13] Y. Pa, S. Suzuki, K. Yoshioka, T. Matsumoto, T. Kasama, and C. Rossow, "IoTPTOT: Analysing the Rise of IoT Compromises," *USENIX Workshop on Offensive Technologies*, 2015.
- [14] S. Srinivasa, J. M. Pedersen, and E. Vasilomanolakis, "Gotta Catch 'Em All: A Multistage Framework for Honeypot Fingerprinting," *Digital Threats: Research and Practice*, 2023.